



Bắt gói tin như chuyên gia — dòng lệnh

Network Thực Chiến · Series: Debug Mạng từ A–Z · Tập 04

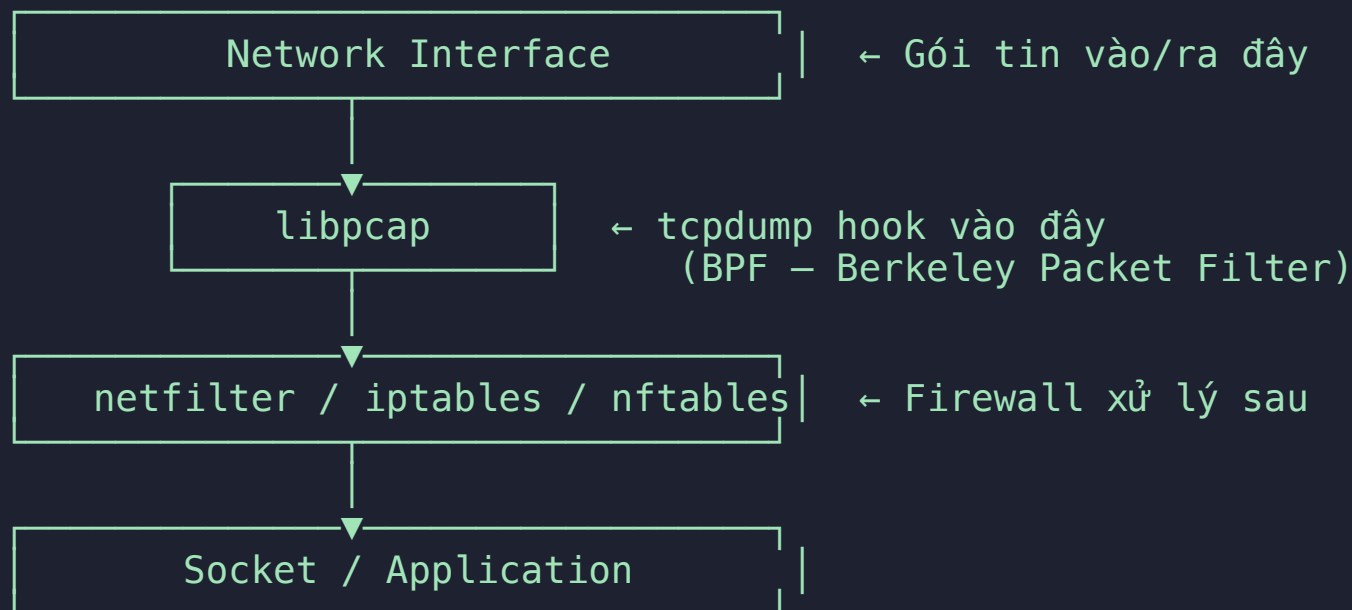
Nội dung

1. **Cách hoạt động** — libpcap, kernel hook, vị trí trong stack
2. **Flags cốt lõi** — Bộ tham số cần nhớ
3. **Cheatsheet** — Filter theo host, port, protocol, boolean
4. **Lưu & đọc file pcap** — Kết hợp với Wireshark
5. **TCP flags filter** — Bắt SYN, RST, debug kết nối
6. **Đọc output tcpdump** — Anatomy từng trường
7. **Kịch bản thực chiến** — 5 tình huống hay gặp nhất

Phần 1

Cách hoạt động

tcpdump hook vào đâu trong network stack?



Ý nghĩa quan trọng:

`tcpdump` bắt gói tin **TRƯỚC** khi firewall xử lý.

Nếu thấy gói tin trong `tcpdump` nhưng app không nhận được → **firewall đang drop**.

Nếu `tcpdump` không thấy gì → vấn đề ở routing, NIC, hoặc phía gửi.

tcpdump dùng BPF — Tại sao nhanh?

BPF (Berkeley Packet Filter) = bộ lọc chạy trong kernel space:

Không dùng BPF (cách cũ):

Kernel copy toàn bộ gói tin → User space → App lọc
→ Copy nhiều, tốn memory, chậm khi traffic cao

Dùng BPF (tcpdump):

Kernel chạy filter ngay trong kernel space
→ Chỉ copy gói tin KHỚP filter lên user space
→ Ít overhead, an toàn khi traffic cao

Thực tế: Có thể chạy `tcpdump` trên production server đang có load — overhead rất thấp nếu filter tốt.

⚠ Không filter (`tcpdump -i eth0` không có điều kiện) = copy toàn bộ traffic → tốn CPU khi high throughput.

Phần 2

Flags & Cheatsheet

Flags cốt lõi

Flag	Ý nghĩa	Ghi nhớ
<code>-i eth0</code>	Interface cần bắt	<code>-i any</code> = tất cả
<code>-n</code>	Không resolve DNS	Nhanh hơn, không noise
<code>-nn</code>	Không resolve DNS lẫn port name	<code>80</code> thay vì <code>http</code>
<code>-v / -vv / -vvv</code>	Verbose — thêm TTL, TOS, decode protocol	<code>-vvv</code> = rất chi tiết
<code>-A</code>	Payload dạng ASCII	Xem HTTP body plaintext
<code>-X</code>	Payload dạng hex + ASCII	Debug binary protocol
<code>-e</code>	In MAC address	Debug L2, ARP, VLAN
<code>-c N</code>	Dừng sau N gói	Giới hạn capture
<code>-w file.pcap</code>	Lưu ra file pcap	Mở Wireshark sau
<code>-r file.pcap</code>	Đọc lại file pcap	Không cần root
<code>-s 0</code>	Capture full packet	Mặc định truncate ở 262144 bytes

Filter: Theo interface

```
sudo tcpdump -i eth0      # Interface cụ thể
sudo tcpdump -i any       # Tất cả interface (bao gồm lo)
sudo tcpdump -i lo        # Loopback – debug IPC / localhost traffic

# Liệt kê interface có thể bắt:
sudo tcpdump -D
```

Khi nào dùng `-i any` ?

- Không biết traffic đi qua interface nào (multi-NIC server)
- Debug container/pod traffic (veth, docker0, cni0...)
- Debug VXLAN / overlay network

⚠ `-i any` không capture ở promiscuous mode — không thấy L2 header.
Dùng `-i eth0` khi cần debug MAC address / VLAN tag.

Filter: Theo host và network

```
sudo tcpdump host 192.168.1.1      # Đến HOẶC từ IP này
sudo tcpdump src host 192.168.1.1  # Chỉ traffic TỪ IP này
sudo tcpdump dst host 192.168.1.1  # Chỉ traffic ĐẾN IP này

sudo tcpdump net 192.168.1.0/24     # Toàn subnet
sudo tcpdump src net 10.0.0.0/8     # Từ subnet 10.x.x.x
```

Filter: Theo port và protocol

```
sudo tcpdump port 80           # HTTP (src hoặc dst)
sudo tcpdump port 443          # HTTPS
sudo tcpdump port 53           # DNS
sudo tcpdump portrange 8080-8090 # Port range

sudo tcpdump tcp port 22        # Chỉ TCP SSH
sudo tcpdump udp port 53        # Chỉ UDP DNS
sudo tcpdump icmp               # Chỉ ICMP (ping)

sudo tcpdump src port 3306      # Traffic từ MySQL
sudo tcpdump dst port 5432      # Traffic đến PostgreSQL
```

Filter: Boolean logic

```
# AND – cả hai điều kiện phải đúng
sudo tcpdump host 10.0.0.1 and port 80
sudo tcpdump tcp and port 443 and dst host 10.0.0.5

# OR – một trong hai
sudo tcpdump host 10.0.0.1 or host 10.0.0.2
sudo tcpdump port 80 or port 443

# NOT – loại trừ
sudo tcpdump not port 22          # Bỏ SSH – giảm noise khi debug
sudo tcpdump not host 192.168.1.1

# Kết hợp phức tạp – dùng ngoặc kép và ngoặc đơn
sudo tcpdump "host 10.0.0.1 and (port 80 or port 443)"
sudo tcpdump "not port 22 and not port 53 and host 10.0.0.5"
```

💡 **Tip:** Thêm `not port 22` khi SSH vào server để debug — tránh bắt traffic SSH của chính mình.

Lưu và đọc file pcap

```
# Lưu vào file – gửi team, mở Wireshark
sudo tcpdump -i eth0 -w /tmp/capture.pcap

# Giới hạn số gói
sudo tcpdump -i eth0 -c 1000 -w capture.pcap

# Rotate file mỗi 60 giây (long-running capture)
sudo tcpdump -i eth0 -G 60 -w /tmp/cap_%Y%m%d_%H%M%S.pcap

# Đọc lại – không cần root
tcpdump -r capture.pcap
tcpdump -r capture.pcap -nn host 10.0.0.1 and port 80

# Remote capture → thẳng vào Wireshark trên máy local
ssh user@server "sudo tcpdump -nn -i eth0 -U -w - port 8080" | wireshark -k -i -
```

-U (unbuffered): ghi ngay ra stdout, không chờ buffer đầy — bắt buộc khi pipe sang Wireshark.

TCP Flags filter — Debug kết nối

```
# Bắt SYN — kết nối mới đang được tạo
sudo tcpdump "tcp[tcpflags] & tcp-syn != 0"

# Bắt RST — kết nối bị reset đột ngột (dấu hiệu lỗi)
sudo tcpdump "tcp[tcpflags] & tcp-rst != 0"

# Bắt SYN thuần — không có ACK (port scan / firewall block)
sudo tcpdump "tcp[tcpflags] == tcp-syn"

# Bắt FIN — kết nối đang đóng
sudo tcpdump "tcp[tcpflags] & tcp-fin != 0"
```

Khi nào dùng TCP flags filter?

Mục tiêu	Filter
Xem kết nối mới đến port 80	"tcp[tcpflags] == tcp-syn" and port 80
Phát hiện connection reset	"tcp[tcpflags] & tcp-rst != 0"
Detect port scan	"tcp[tcpflags] == tcp-syn" and not port 22
Debug half-open connections	SYN nhiều, SYN-ACK ít

Phần 3

Đọc output tcpdump

Anatomy của một dòng output

```
14:23:45.123456 IP 10.0.0.5.54321 > 10.0.0.10.80: Flags [S], seq 1234567, win 65535, length 0
```

Phần	Ý nghĩa
14:23:45.123456	Timestamp — microsecond precision
IP	Protocol L3: IP , IP6 , ARP , ICMP ...
10.0.0.5.54321	Source: IP.Port
>	Hướng gói tin
10.0.0.10.80	Destination: IP.Port
Flags [S]	TCP flags (xem bảng bên dưới)
seq 1234567	Sequence number
win 65535	TCP receive window size
length 0	Payload size (0 = header only, không có data)

TCP Flags — Đọc trạng thái kết nối

Flag	Ký hiệu	Ý nghĩa	Khi nào xuất hiện
SYN	[S]	Mở kết nối	Bắt đầu 3-way handshake
SYN-ACK	[S.]	Server đồng ý	Handshake bước 2
ACK	[.]	Xác nhận	Sau mỗi data segment
PSH-ACK	[P.]	Gửi data	HTTP request/response
FIN-ACK	[F.]	Đóng kết nối	TCP teardown
RST	[R]	Reset đột ngột	Lỗi — cần điều tra
RST-ACK	[R.]	Reset + xác nhận	App từ chối kết nối

Đọc 3-way handshake trong tcpdump:

```
client > server: Flags [S]           ← SYN
server > client: Flags [S.]          ← SYN-ACK
client > server: Flags [.]           ← ACK
client > server: Flags [P.]          ← Data (HTTP request)
server > client: Flags [P.]          ← Data (HTTP response)
server > client: Flags [F.]          ← FIN
```


RST — Tín hiệu cần điều tra ngay

```
# Kịch bản: Kết nối bị reset
10.0.0.5.52341 > 10.0.0.10.8080: Flags [S]    ← Client gửi SYN
10.0.0.10.8080 > 10.0.0.5.52341: Flags [R.]  ← Server RST ngay!
```

RST xuất hiện khi:

Nguyên nhân	Dấu hiệu trong tcpdump
Port không listen	SYN → RST ngay lập tức
Firewall reject (REJECT rule)	SYN → RST từ firewall host
App crash giữa chừng	Data → RST đột ngột
Load balancer timeout	Sau khoảng thời gian idle
Kernel reject kết nối	RST không có SYN trước

⚠ **RST khác DROP:** *DROP* = gói tin biến mất (timeout ở client). *RST* = client nhận thông báo từ chối ngay.

Phần 4

Kịch bản thực chiến

Scenario A: "Traffic có vào server không?"

Vấn đề: App không nhận request, không biết traffic có đến server không.

```
# Terminal 1: Bắt traffic trên port app
sudo tcpdump -nn -i any port 8080

# Terminal 2: Gửi request từ client
curl http://server_ip:8080/health
```

Đọc kết quả:

```
# Thấy SYN và SYN-ACK → Kết nối vào được, vấn đề ở app
10.0.0.5.52341 > server.8080: Flags [S]
server.8080 > 10.0.0.5.52341: Flags [S.]

# Thấy SYN nhưng không thấy SYN-ACK → firewall DROP ở OUTPUT chain
10.0.0.5.52341 > server.8080: Flags [S]
[... không có phản hồi]

# Không thấy gì → gói tin không đến interface
# → Vấn đề: routing sai, IP sai, firewall ở upstream DROP
```

Scenario B: "Verify firewall rule hoạt động đúng"

```
# Terminal 1: Bắt traffic đến port cần kiểm tra  
sudo tcpdump -nn -i eth0 port 80
```

```
# Terminal 2: Gửi request  
curl http://server_ip/test
```

Matrix chẩn đoán:

tcpdump thấy	Kết quả kết nối	Kết luận
SYN + SYN-ACK + data	Thành công	Firewall OK 
SYN nhưng không SYN-ACK	Timeout	Firewall DROP ở INPUT → OUTPUT
SYN + RST ngay	Connection refused	Firewall REJECT hoặc port không listen
Không thấy gì	Timeout	Gói tin bị drop trước khi vào NIC

💡 Nhớ: `tcpdump` bắt **TRƯỚC** firewall → thấy SYN = gói đã vào NIC.
Không thấy SYN-ACK = kernel (hoặc firewall OUTPUT chain) đang drop.

Scenario C: "Debug TLS handshake failure"

```
# Capture TLS handshake – dù không decrypt content, vẫn thấy:  
# ClientHello, ServerHello, Certificate, Alert (lỗi)  
sudo tcpdump -nn -i any port 443 -v -w tls_debug.pcap  
  
# Mở trong Wireshark:  
# Statistics → Expert Information → lọc theo "Alert"  
# Analyze → Decode As → TLS (nếu dùng port khác 443)
```

Thấy gì trong pcap khi TLS fail:

```
# ClientHello – client gửi cipher suites hỗ trợ  
client > server: Flags [P.] ... TLSv1.3 Client Hello  
  
# Alert – server báo lỗi  
server > client: Flags [P.] ... TLSv1.3 Alert  
    Level: Fatal, Description: certificate_unknown  
                                handshake_failure  
                                unknown_ca
```

Dùng Wireshark filter: `tls.alert_message.desc` để xem mã lỗi TLS cụ thể.

Scenario D: "Debug DNS — Query có đến resolver không?"

```
# Bắt toàn bộ DNS traffic
sudo tcpdump -nn -i any port 53 -v

# Output mẫu:
# Query:
# 10.0.0.5.54321 > 8.8.8.8.53: A? google.com. (28)
#
# Response:
# 8.8.8.8.53 > 10.0.0.5.54321: A google.com. 142.250.196.46 (44)
```

Checklist debug DNS:

```
# App có gửi query không?
sudo tcpdump -nn -i any port 53 and src host <app_ip>

# Resolver có trả lời không?
sudo tcpdump -nn -i any port 53 and dst host <app_ip>

# Query đến đúng resolver chưa?
sudo tcpdump -nn -i any port 53 -v | grep ">"
```

Scenario E: "Remote capture → Wireshark trực tiếp"

Khi server không có GUI, không muốn lưu file pcap rồi copy về:

```
# Trên máy local – một lệnh duy nhất  
ssh user@server "sudo tcpdump -nn -i eth0 -U -w - port 8080" | wireshark -k -i -
```

Giải thích:

Phần	Ý nghĩa
<code>ssh user@server</code>	Kết nối SSH đến server
<code>tcpdump -U -w -</code>	<code>-U</code> : unbuffered, <code>-w -</code> : ghi ra stdout
<code> </code>	Pipe output SSH về máy local
<code>wireshark -k -i -</code>	<code>-k</code> : mở ngay, <code>-i -</code> : đọc từ stdin

⚠ Cần cài Wireshark trên máy local. Pipe qua SSH không mã hoá thêm — traffic tcpdump đi qua SSH tunnel đã mã hoá.

Key Takeaways

Bộ lệnh cốt lõi:

```
# Xác định traffic có vào không
sudo tcpdump -nn -i any port 8080
# Xem HTTP body (plaintext)
sudo tcpdump -A -s 0 port 80 | grep -E "GET|POST|Host:|HTTP/"
# Lưu pcap để phân tích sau
sudo tcpdump -nn -i eth0 -w /tmp/capture.pcap host 10.0.0.1
# Remote capture → Wireshark
ssh user@server "sudo tcpdump -nn -i eth0 -U -w - port 8080" | wireshark -k -i -
# Bắt RST – phát hiện kết nối bị reset
sudo tcpdump -nn "tcp[tcpflags] & tcp-rst != 0"
```

Thấy trong tcpdump	Kết luận
SYN nhưng không SYN-ACK	Firewall DROP
SYN → RST ngay	Port không listen hoặc firewall REJECT
Không thấy gì	Vấn đề routing / NIC / upstream
Handshake OK, app không response	Vấn đề application layer
[R] giữa chừng	App crash / timeout / load balancer

Cảm ơn đã theo dõi! 🙏

Network Thực Chiến

Tập tiếp theo: **iperf3** — Đo băng thông thực tế

"Nếu *tcpdump* thấy *SYN* nhưng không thấy *SYN-ACK* — *firewall* đang drop."